

Student Onramp Feedback — "From the map to the tidal signal" v1 (slice run)

AI-generated draft (Claude, Anthropic) — compiled from onramp improvement notes.

Student: Connor. Session date: 2026-06-16.

What worked well

- Step 3 (Get the data) — reported as about right, with no friction
- Step 4 (See the science) — appropriately challenging given it combines pandas, timeseries, and tidal science; no changes suggested

PART 1: STUDENT-SUGGESTED IMPROVEMENTS (session feedback, verbatim intent)

Step 0 — Get to the data

- Add more focus on shell navigation in the guided exercise. The current exercise is thin; students who are new to the terminal would benefit from more hands-on practice with `pwd/cd/ls/head` before the capstone.

Step 1 — Locate it

- The exercise was slightly too easy relative to the volume of content (three full chapters: JupyterLab, Python fundamentals, NumPy/matplotlib). Add more exercise questions to verify each chapter, not just the final matplotlib plot.

Step 2 — Why it's there

- Explicitly contrast decompression melting and the hotspot mechanism. The two feel vaguely similar (both produce magma) but the cause is different (pressure drop vs. thermal plume). Students need a clear side-by-side before the capstone asks them to connect both to eruptions.

Step 3 — Get the data

- No suggestions. Student reported about right, no friction.

Step 4 — See the science

- No specific suggestions. Student noted it was appropriately harder than earlier steps given it combines pandas + timeseries + tidal science.

PART 2: SUGGESTED IMPROVEMENTS FOR ASSESSMENT ROBUSTNESS:

The following are structural vulnerabilities observed during the run that could allow a student to receive an inaccurate placement or bypass the intent of the assessment.

1. "I don't know, move on" bypass

PROBLEM: A student can short-circuit the full hint ladder by preemptively saying "I don't know, move on" after the first rung (or before any hint).

The procedure instructs walking the ladder before scoring wrong, but a student who wants full scaffold for everything can skip R2 and R3.

SUGGESTION: Require at least R2 engagement before recording a probe as wrong. If the student explicitly refuses R2, note the refusal in the progress.md record so a human reviewer can flag it. Alternatively, add language in the welcome that skipping hints scores the same as wrong and does not change the treatment in a meaningful way (full scaffold is still full scaffold — there's no benefit to gaming down).

2. Single probe covers three skills (step 1)

PROBLEM: p02_matplotlib_bathy is the sole probe for jupyter_basics, python_fundamentals, AND matplotlib_plot. If a student blanks on that one probe, all three skills are scored Novice regardless of their actual

Python or Jupyter knowledge. A student who knows Python well but has never used matplotlib gets the same placement as one who has never opened a notebook.

SUGGESTION: Add at least one probe per distinct skill (a Jupyter-specific probe and a Python-fundamentals probe separate from the matplotlib probe).

3. p00c_shell_peek scored wrong but head is a gating objective

PROBLEM: head/cat is listed as a gating objective for unix_basics, but the probe (p00c_shell_peek) is the only place it is assessed. A student who cannot peek at a file without an editor is still placed Novice (same as one who can't navigate at all), so the distinction is invisible in the treatment. In the capstone the student was never required to use head.

SUGGESTION: Include head explicitly in the step 0 exercise and capstone (e.g., "peek at the first 5 lines of plate_boundary.csv before opening it in vi"). This makes the skill load-bearing, not just assessed.

4. vi probe and capstone are weakly coupled

PROBLEM: p00b_vi_edit can be scored wrong even if a student completes the capstone correctly (as happened here). The gate checks p00b not wrong, but a student who learns vi from the Wikibooks reference and passes the capstone should clear the gate regardless of the pre-teaching probe score. The gate condition "(p00 not wrong; p00b not wrong)" conflates the pre-teaching assessment with the post-teaching check.

SUGGESTION: Rewrite the gate condition to evaluate the capstone demonstration directly rather than referencing probe scores. For example: "Student opened a file in vi and exited with :q! or :q" is verifiable from the capstone; the probe score is the pre-teaching baseline, not the post-teaching gate.

5. Data files absent in development/test runs

PROBLEM: When staged data files are missing, all capstones that require real file I/O (steps 0, 1, 3, 4) must be adapted on the fly. The student adapts correctly, but the tutor cannot verify actual file loading, real dataset dims/coords, or true tidal signal extraction.

SUGGESTION: Stage minimal synthetic data files (even 10-row CSVs and small NetCDFs) in the repo as fixtures for development/testing runs.

This keeps capstones testable without requiring the full OOI payload.

6. Step 1 exercise depth vs. chapter breadth mismatch

PROBLEM: Three chapters are assigned (JupyterLab, Python fundamentals, NumPy/matplotlib) but only one exercise question bridges all three into a single matplotlib task. A student who skimmed the chapters can still pass. Confirmed by student feedback ("slightly too easy").

SUGGESTION: Add two intermediate exercise questions — one targeting Python fundamentals (e.g., indexing, slicing, calling a function) and one targeting JupyterLab specifically (e.g., what does restarting the kernel do? what is the difference between a code cell and a markdown cell?) — before reaching the matplotlib capstone.

7. Assessment does not probe pandas or xarray until after the chapter reads

PROBLEM: The initial assessment probes p04 (xarray) and p05 (pandas) before any teaching. Students with no background predictably score wrong, land at Novice, and get full scaffold. This is correct behavior, but it means the probes add friction for new students without adding information — the outcome is the same regardless of whether they blank or struggle.

SUGGESTION: For clearly introductory tools (xarray, pandas), consider

framing the probe as "have you used this library before?" plus a short task, with the explicit option to say "never heard of it" without the hint ladder walk. Reserve the ladder for skills where partial knowledge is plausible (shell, Python basics, lat/lon).